工程入門 / 01

先把格式工程做成可用工具

化學結構格式偵測與轉換工程入門

RDKit 實作與 torch molecule 學習路線

版本 0.1.0　　2026 年 11 月 6 日　　適合零基礎學生

你的第一個目標，是讓使用者上傳小分子結構檔案後，得到可檢查的結構圖、其他結構檔與性質表。本指南把這個目標拆成可以執行、可以解釋、可以驗證的工程流程。先完成此流程，再決定是否需要機器學習模型。

元件	在本題負責甚麼
RDKit	讀寫分子、檢查價態、畫結構圖、計算描述符；第一版核心。
torch-molecule	用訓練好的模型做性質預測、分子表示學習或生成。
Torch Molecule Studio	使用者操作的應用層；真正原始碼取得後才可確認整合方式。

這次交付可以做甚麼

ChemBridge 是獨立可執行的原型，附有本機網頁介面、CLI、原始碼、範例、測試與 DeepSeek 提示詞。核心轉換可在 CPU 執行，不需要 LLM 金鑰。它尚未整合到 Studio，也未完成任意化學文件或圖片辨識。

閱讀與操作順序

頁次	你要完成的事
2–3	安裝、跑乙醇範例，理解文字如何變成分子圖。
4–6	看格式偵測、錯誤處理、輸出與核心程式。
7–8	理解 torch-molecule，學會展示工程證據。
9–11	安排下一週工作，使用 DeepSeek 提示詞，建立 GitHub 並推送。

在 Windows 跑出第一個結果

建議使用 Python 3.12 的 64-bit 版本，另建環境，保留你之前的專案環境。解壓 ZIP 後，進入 chembridge_mvp 資料夾，在檔案總管的地址列輸入 powershell 並按 Enter。

先確認 Python

```
py -3.12 --version
```

若找不到此版本，先安裝 Python 3.12，重新開啟 PowerShell。以下命令都在包含 README.md 和 app.py 的專案根目錄執行，逐行貼上。

```
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements-ui.txt
.\.venv\Scripts\python.exe scripts\first_steps.py
.\.venv\Scripts\python.exe scripts\run_demo.py
```

first_steps 會建立 learning_output 資料夾。run_demo 會顯示一個 runs/demo_時間 的新資料夾；每次執行都建立新目錄。

開啟本機操作介面

```
.\.venv\Scripts\python.exe -m streamlit run app.py
```

依終端機顯示的地址在瀏覽器開啟頁面，通常是 http://localhost:8501。選擇 examples/molecules.csv，按「偵測並轉換」，檢查圖片和性質，再下載結果 ZIP。要結束服務，回終端機按 Ctrl+C。

你看到的情況	如何處理
找不到 app.py 或 requirements	目前資料夾不對；回到解壓後的 chembridge_mvp。
pip 安裝失敗	保留完整錯誤，確認 Python 版本、網路與 64-bit 環境。
Output directory already exists	CLI 的 --out 請改用新名稱，避免覆寫已有結果。
混合壞資料出現 failed/partial	先看 errors.json；這可能正是預期的錯誤報告。

首次安裝需要下載第三方依賴；ZIP 沒有內附 Python 或虛擬環境。完成安裝後，核心轉換不需網路。start_windows.bat 提供另一個啟動入口；Windows 實際操作仍需在你的電腦驗證。

理解一個分子如何變成輸出

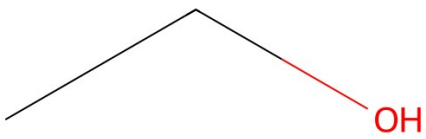
名詞	白話解釋
SMILES	用文字記錄原子和連接關係；CCO 表示乙醇。
RDKit Mol	電腦內的分子圖，包含原子、鍵及可用的結構資訊。
CSV	表格容器；必須找出其中真正存放結構的欄位。
描述符 descriptor	由結構計算或估計的數值，如分子量、HBD、TPSA。

最小核心程式

```
from rdkit import Chem
from rdkit.Chem import Descriptors, Draw, Lipinski

mol = Chem.MolFromSmiles("CCO")
assert mol is not None
print(Descriptors.MolWt(mol))
print(Lipinski.NumHDonors(mol))
Draw.MolToImage(mol).save("ethanol.png")
```

第一行轉換不是畫圖，而是先讀懂分子。assert 確認沒有解析失敗。接著才可計算分子量、數氫鍵供體、畫圖。完整可執行範例在 scripts/first_steps.py。



本工具實際生成的乙醇 2D 結構圖。圖上省略部分碳與氫的標示，是化學骨架圖的慣例，不代表分子少了這些原子。

已知乙醇結果：分子式 C2H6O；分子量約 46.069 Da；HBD=1；HBA=1。你應能指著程式說明這些數字如何來，並把 CCO 改成 CCN 再觀察差異。

讓系統知道正在讀甚麼

可靠的自動偵測要同時看內容與解析結果。副檔名可以提供線索，但使用者可能改錯檔名；CSV 也可能同時含兩種分子表示。

順序	實際策略
1 內容標記	TRIPOS、PDB records、SDF 的 $$$$、MOL 的版本標記與 M END。
2 表格與 schema	辨認 CSV/TSV 分隔符、JSON 容器，再核對結構欄位。
3 文字與副檔名	InChI= 前綴、SMILES/SELFIES 檔名；不足時試解析。
4 明確拒絕歧義	多個結構欄、含糊的括號文字或不明 schema，需要使用者指定。

核心程式在哪裡

chembridge/io.py 的 detect() 選讀取方式；parse_records() 逐筆建分子。_table_records() 處理表格欄位。下列片段展示為何兩個候選欄位不會被默默選掉。

```
elif len(named) > 1:
    raise InputError("Multiple structure columns: "
        + ", ".join(h for h, _ in named)
        + ". Select --column (and optionally --column-format).")
```

你可以重現的三個案例

範例	預期結果
molecules.csv	偵測 CSV，使用 SMILES 欄，5 筆成功。
misnamed.csv	內容是 MOL；偵測 MOL 並記錄副檔名衝突。
ambiguous_columns.csv	同時有 smiles/inchi；先要求選欄，選 smiles 後成功。

未知欄名會取樣前 20 筆，以可解析率尋找唯一候選欄；80% 是本工具規則，不是模型信心。只含分子式、名稱或 CAS 的表格通常不足以唯一還原結構，不能強行當成 SMILES。

SELFIES 與某些括號 SMILES 可能看起來相同；請明確指定格式。自動偵測是一套有範圍的工程規則，並非能讀懂任意化學文件。

讀懂輸出與資訊保留

輸出	要看甚麼
properties.csv	可用 Excel 查看各分子的性質；欄位附單位。
molecules.json	成功分子的結構、性質、來源 metadata、警告及檔案路徑。
PNG 與 SVG	預設前 100 個成功分子的 2D 圖；用副本繪製。
MOL SDF SMI InChI SELFIES	交給其他化學工具使用；各格式能承載的資訊不同。
errors 與 export_errors	原文、來源記錄索引及失敗原因；不要只檢查成功圖像。
manifest 與 original_input	偵測理由、版本、計數、SHA-256、耗時，以及原始輸入。

兩個成功分子的實際數值

性質	乙醇	阿士匹靈
分子式	C2H6O	C9H8O4
分子量 Da	46.069	180.159
HBD	1	1
HBA	1	3
TPSA Å^2	20.23	63.60
QED	0.407	0.550

HBD 是氫鍵供體計數；HBA 是受體計數。TPSA 描述極性表面積；RDKit logP 是模型估計。QED 是 drug-likeness 描述，不能當作藥效、安全性或能否成藥的結論。

甚麼會保留 甚麼不能保證

本版不去鹽、不中和、不移除重複分子、不做互變異構體標準化。SDF/MOL 有原始座標時保留；無座標時生成 2D。SMILES 不儲存原始 3D 座標，標準 InChI 亦可能正規化部分資訊。原檔備份與 JSON metadata 有助追溯，但不能把所有格式轉換稱為普遍無損。

跟著核心程式看完整邏輯

檔案與函式	負責甚麼
io.py load_text decode_bytes	讀取文字、BOM 編碼及 gzip，限制檔案大小。
io.py detect	選擇解析器，回傳理由、方法與警告。
io.py parse_records	逐筆解析與驗證；失敗資料仍保留。
pipeline.py properties	計算描述符，沒有呼叫 LLM 或訓練模型。
pipeline.py convert_file	串起流程、寫出檔案、記錄 manifest。
__main__.py 與 app.py	CLI 與本機介面，共用同一核心流程。

讀取後才產生標準表示及數值

```
mol = record.mol
smiles = Chem.MolToSmiles(
    mol, canonical=True, isomericSmiles=True)
props = properties(mol)
export_mol = Chem.Mol(mol)
```

isomericSmiles=True 讓可用的立體與同位素資訊進入輸出。建立副本是為了讓匯出和畫圖的座標操作不改動原始分子。Canonical 字串是 RDKit 的正規表示，不能取代完整化學身分政策。

日後給 Studio 呼叫的入口

```
from chembridge import convert_file
manifest = convert_file(
    "uploaded_file.sdf", "new_job_directory",
    outputs="json,csv,png,sdf")
```

Studio 的後端只需重用此入口，前端顯示偵測、選欄、結果與錯誤。這是可整合的界面；真正 Studio 的路由、登入、資料儲存與框架，必須讀到其原碼後才能修改。

完整來源記錄數應等於成功數加失敗數。CLI 退出碼 0 表示全部成功；1 表示資料／輸出有失敗且已產生報告；2 表示輸入、設定或檔案層級問題。

用這個題目學 torch molecule

torch-molecule 的價值在模型工作流程。它接受分子表示，處理圖特徵、訓練或載入模型，再輸出預測、向量或新分子。你不必先學完所有模型，才開始做檔案轉換。

目錄	初學者先看甚麼
base	模型共同設定、device、保存／載入及 fitted 檢查。
predictor	先看 GNNMolecularPredictor 的 fit() 與 predict()。
utils/graph	graph_from_smiles：原子特徵、連接關係、鍵特徵。
datasets	讀取目標與來源，理解切分；標籤和單位不可省略。
encoder generator nn	向量表示、分子生成與神經網路元件，後續再深入。

圖模型讀甚麼

graph_from_smiles() 走訪每個原子與每條鍵。N 個原子對應 N 列原子特徵；E 條化學鍵加入兩個方向，所以 edge_index 的形狀是 [2, 2E]。這個圖資料不是 PNG 圖片。

先明確定義預測任務

需要預測甚麼？標籤從哪裡來？單位是甚麼？checkpoint 適用哪些分子？只有這些確定，才應接模型。本包 model_adapter.py 只是把成功 SMILES 依 ID 順序交給已有 predictor，未執行真正模型推論。

切分方式	你要理解的差別
random	隨機分列；重複結構可能跨集合。
scaffold	依骨架分組，檢查對未見骨架的泛化。
butina	依指紋相似度聚類，整組分配。
size	按重原子數分開，檢查尺寸外推。

此次上游快照的 train_test_split 回傳兩份 train/holdout。若 holdout 用來選模型，就應另留最終 test；不能用同一份報告「未碰過」的測試分數。詳細讀碼順序與固定 commit 在 TORCH_MOLECULE_SOURCE_MAP_zh.md。

用證據向教授展示工程理解

交付前在 Linux／Python 3.12.14、RDKit 2025.09.4、SELFIES pip 2.2.0、Streamlit 1.51.0 下實測，39 項測試通過。SELFIES 模組自報版本為 2.1.1，因此執行紀錄同時保留 distribution 與 module 版本。

驗證內容	**證據與限制**
基本轉換	MOL V2000/V3000、SDF、CSV/TSV、SMILES、InChI、SELFIES、JSON、gzip 的固定範例。
可靠性	錯副檔名、歧義欄、缺值、壞 JSON、錯欄數、覆寫拒絕。
化學資訊	測試選定立體化學、同位素、電荷、鹽與 3D 座標往返。
本機介面	AppTest 載入與結果顯示測試；非實際瀏覽器上傳下載驗收。
模型介面	fake predictor 測順序；非真正 torch-molecule 模型效果。

三個可直接展示的結果

檔案	**實際結果**
molecules.csv	CSV；5 成功、0 失敗。
mixed_invalid.csv	CSV；1 成功、2 失敗，錯誤原文保留。
misnamed.csv	MOL；1 成功，記錄與副檔名衝突。

三分鐘示範腳本

第 1 分鐘：打開 molecules.csv，指出 SMILES 欄；展示乙醇圖片與 46.069 Da，連到 MolFromSmiles 和 properties()。

第 2 分鐘：打開 mixed_invalid.csv、errors.json 和 manifest；指出哪一列失敗、為何不應偷偷丟棄。

第 3 分鐘：展示錯名檔與 ambiguous_columns.csv；說明內容優先與要求選欄，最後指出 Studio 尚待整合。

測試紀錄和 demo 產物位於 evidence。這些測試證明選定案例可重現，不能當成真實世界格式偵測準確率、完整保真率或 Windows 實測。

安排下一週與閱讀次序

日次	行動與具體產物
1	跑 first_steps；自己解釋 Mol、分子圖、圖片與描述符。
2	跑三個 demo，逐一打開輸入、核心函式和輸出。
3	收集 10–20 個真實工作樣本，人工標記期望格式與欄位。
4	分析最常見失敗，修正一個缺口並新增回歸案例。
5	取得真正 Studio repo 或 ZIP，確認上傳入口與後端框架。
6	整合一次上傳、預覽、選欄、轉換、下載工作流程。
7	建立新 GitHub 倉庫並推送，從全新 clone 重跑 README。

先讀文件 再讀基礎論文

優先閱讀 RDKit Getting Started 的 Reading, Drawing, and Writing Molecules，立即手動重現，再按源碼地圖看 torch-molecule 的 graph_from_smiles 和 GNN predict。

Open Babel 2011 論文幫你理解格式互轉的既有工作；SELFIES 2023 library 論文幫你理解編碼與限制；QED 2012 論文解釋 drug-likeness 指標。它們是基礎閱讀，不是最新趨勢綜述。完整來源與固定 commit 見 SOURCES.md。

```
RDKit
https://www.rdkit.org/docs/GettingStartedInPython.html
Open Babel paper
https://pmc.ncbi.nlm.nih.gov/articles/PMC3198950/
SELFIES library paper
https://arxiv.org/abs/2302.03620
```

工程成果怎樣走向研究

格式互轉已有成熟工具；目前優先把 Studio 需要的流程做可靠。若再做論文，可研究歧義與拒絕、轉換資訊損失、不完整結構的恢復。必須再做新穎性審查、比較基線及真實資料實驗，不能把原型包裝成已證明的頂會創新。

把工作交給 DeepSeek 繼續執行

建議把整個 ZIP 交給具有檔案與終端機能力的 coding agent。一般 DeepSeek 聊天若沒有這些能力，只能協助讀碼與指導操作，不能因為它回覆「完成」就當作真的執行過。

可以直接複製的短提示詞

請解壓並讀取 chembridge_mvp，先讀 README、docs、chembridge、app.py、scripts 和 tests。我的目標是替 Torch Molecule Studio 做常見小分子結構格式的偵測、轉換、圖片和性質輸出。我是 Windows 零基礎學生。

請另建 Python 3.12 環境，安裝 requirements-ui.txt 和 requirements-dev.txt，實際執行 first_steps、run_demo 與 pytest。記錄版本、命令、退出碼及真實結果；修正失敗後再驗證。開啟介面實際上傳、選欄、下載；檢查一般 CSV、混合壞資料、錯誤副檔名及兩個結構欄位。

不要默默中和、去鹽或刪除壞資料；不確定就要求指定欄位。不要把 RDKit 性質當成模型預測、把 fake predictor 當真正推論、把測試通過率當格式辨識準確率。

確認真正 Studio 的 repo、入口與資料契約後，重用 convert_file 整合上傳→預覽→選欄→轉換→下載。若缺真正原碼，先完成獨立工具驗收，再列出整合欠缺資料，不能用第三方套件 fork 冒充 Studio。

最後建立新 GitHub 倉庫 chem-structure-converter，完成 commit 與 git push，核對遠端 SHA，再全新 clone 驗證。給我可重跑程式、中文邏輯鏈、核心片段、真實紀錄、repo URL 及未完成項目。

完整提示詞與回傳格式

更完整的逐階段任務已放在 prompts/DEEPSEEK_EXECUTION_PROMPT_zh.txt。請優先使用該檔，避免漏掉格式限制、真實資料驗收與 Studio 原碼核對。

完成後使用 prompts/RETURN_RESULTS_TO_CHATGPT_zh.txt 填入環境、命令、測試、demo、介面操作、修改檔案、Studio 路徑及你不理解的地方，再交回來逐項檢查。

你現在的第一步

先解壓、完成第 2 頁安裝、跑出第 3 頁乙醇圖片。當你能清楚解釋「為甚麼先建 Mol 再畫圖」及「壞資料去哪裡」，就已開始掌握這項工程工作的核心。

建立新 GitHub 倉庫並完成推送

新倉庫建議名稱為 chem-structure-converter，應用名稱保留 ChemBridge。建立 repo 與 git push 是 DeepSeek 執行計劃中的必要交付；不是只回傳命令即可。完整步驟在 docs/GITHUB_DELIVERY_zh.md。

先確認帳號與準備提交

```
gh auth status
gh api user --jq .login
git init
git branch -M main
git add .gitignore README.md pyproject.toml requirements*.txt
git add chembridge app.py start_windows.bat scripts tests
git add examples docs prompts evidence
git diff --cached --stat
git diff --cached
```

預期使用 CHANSingYeungSunny 帳號，執行前核對實際登入者及專案位置。檢查 staged files，只納入程式、合成範例與適合分享的文件證據；排除 .env、金鑰、虛擬環境及私人資料。Git author 使用本人設定，不得編造電子郵件。

建立遠端並推送

```
git commit -m "Add chemical structure conversion MVP"
$repo = "CHANSingYeungSunny/chem-structure-converter"
gh repo create $repo --private --source . --remote origin
git push -u origin main
gh repo view --json nameWithOwner,url,isPrivate
git rev-parse HEAD
git ls-remote origin refs/heads/main
```

預設 private；若已有明確公開要求則採用 public。名稱被佔用時不可覆寫不相關 repo，可建立日期尾綴的新名稱。記錄真實 URL、visibility、branch 與 commit SHA，本機 HEAD 要和遠端 main 一致。

全新 clone 是最後驗收

到另一個新目錄 clone，依 README 重建環境、跑 demo 和 pytest，再記錄結果。原工作區成功不等於遠端可重現。執行者須回傳 clone 後的真實證據。

本次交付環境沒有 GitHub CLI 或可用的 GitHub 連接，因此尚未建立遠端 repo 或 push。檔案與提示詞已備妥；請由已登入 GitHub 的本機 coding agent 完成此階段並回傳實際網址。